

```
int main() {
    int T;
    scanf("%d", &T);
    while(T--) {
        memset(buf, 0, sizeof(buf));
        cnt = 0;
        for(int i = 0; i < 2; i++) {
            scanf("%s", s);
            int p = 0;
            draw(s, p, 0, 0, len);
        }
        printf("There are %d black pixels.\n", cnt);
    }
    return 0;
}
```

6.4 图

图（Graph）描述的是一些个体之间的关系。与线性表和二叉树不同的是：这些个体之间既不是前驱后继的顺序关系，也不是祖先后代的层次关系，而是错综复杂的网状关系。

6.4.1 用 DFS 求连通块

例题 6-12 油田（Oil Deposits, UVa 572）

输入一个 m 行 n 列的字符矩阵，统计字符“@”组成多少个八连块。如果两个字符“@”所在的格子相邻（横、竖或者对角线方向），就说它们属于同一个八连块。例如，图 6-9 中有两个八连块。

```
****@
*@@*@
*@**@
@@@*@
@@**@
```

图 6-9 八连块

【分析】

和前面的二叉树遍历类似，图也有 DFS 和 BFS 遍历。由于 DFS 更容易编写，一般用 DFS 找连通块：从每个“@”格子出发，递归遍历它周围的“@”格子。每次访问一个格子时就给它写上一个“连通分量编号”（即下面代码中的 idx 数组），这样就可以在访问之前检查它是否已经有了编号，从而避免同一个格子访问多次：

```

#include<stdio>
#include<cstring>
const int maxn = 100 + 5;

char pic[maxn][maxn];
int m, n, idx[maxn][maxn];

void dfs(int r, int c, int id) {
    if(r < 0 || r >= m || c < 0 || c >= n) return; // “出界”的格子
    if(idx[r][c] > 0 || pic[r][c] != '@') return; //不是“@”或者已经访问过的格子
    idx[r][c] = id; //连通分量编号
    for(int dr = -1; dr <= 1; dr++)
        for(int dc = -1; dc <= 1; dc++)
            if(dr != 0 || dc != 0) dfs(r+dr, c+dc, id);
}

int main() {
    while(scanf("%d%d", &m, &n) == 2 && m && n) {
        for(int i = 0; i < m; i++) scanf("%s", pic[i]);
        memset(idx, 0, sizeof(idx));
        int cnt = 0;
        for(int i = 0; i < m; i++)
            for(int j = 0; j < n; j++)
                if(idx[i][j] == 0 && pic[i][j] == '@') dfs(i, j, ++cnt);
        printf("%d\n", cnt);
    }
    return 0;
}

```

上面的代码用一个二重循环来找到当前格子的相邻 8 个格子，也可以用常量数组或者写 8 条 DFS 调用，读者可以根据自己的喜好选用。这道题目的算法有个好听的名字：种子填充（floodfill）。有兴趣的读者还可以看看维基百科^①中的动画，对 DFS 和 BFS 实现的种子填充有一个更直观的认识。

提示 6-21：图也有 DFS 遍历和 BFS 遍历，其中前者用递归实现，后者用队列实现。求多维数组连通块的过程也称为种子填充（floodfill）。

例题 6-13 古代象形符号（Ancient Messages, World Finals 2011, Uva 1103）

本题的目的是识别 3000 年前古埃及用到的 6 种象形文字，如图 6-10 所示。

^① <http://en.wikipedia.org/wiki/Floodfill>.

					
Ankh	Wedjat	Djed	Scarab	Was	Akeht

图 6-10 古代象形符号

每组数据包含一个 H 行 W 列的字符矩阵 ($H \leq 200$, $W \leq 50$)，每个字符为 4 个相邻像素点的十六进制 (例如, 10011100 对应的字符就是 9c)。转化为二进制后 1 表示黑点, 0 表示白点。输入满足:

- ❑ 不会出现上述 6 种符号之外的其他符号。
- ❑ 输入至少包含一个符号, 且每个黑像素都属于一个符号。
- ❑ 每个符号都是一个四连块, 并且不同符号不会相互接触, 也不会相互包含。
- ❑ 如果两个黑像素有公共顶点, 则它们一定有一个相同的相邻黑像素 (有公共边)。
- ❑ 符号的形状一定和表 6-9 中的图形拓扑等价 (可以随意拉伸但不能拉断)。

要求按照字典序输出所有符号。例如, 图 6-11 中的输出应为 AKW。



图 6-11 输出 AKW

【分析】

“随意拉伸但不能拉断”是一个让人头疼的条件。怎么办呢? 看来不能拘泥于细节, 而要从全局考虑, 找到一个易于计算, 而且在“随意拉伸”时还不会改变的“特征量”, 通过计算和比较“特征量”完成识别。题目说过, 每个符号都是一个四连块, 即所有黑点都连在一起, 而中间有一些白色的“洞”。数一数就能发现, 题目表中的 6 个符号从左到右依次有 1, 3, 5, 4, 0, 2 个洞, 各不相同。这样, 只需要数一数输入的符号有几个“白洞”, 就能准确地知道它是哪个符号了。

6.4.2 用 BFS 求最短路

假设有一个网格迷宫, 由 n 行 m 列的单元格组成, 每个单元格要么是空地 (用 1 来表示), 要么是障碍物 (用 0 来表示)。如何找到从起点到终点的最短路径?

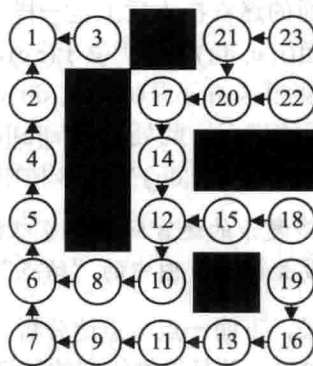
还记得二叉树的 BFS 吗? 结点的访问顺序恰好是它们到根结点距离从小到大的顺序。类似地, 也可以用 BFS 来按照到起点的距离顺序遍历迷宫图。

例如, 假定起点在左上角, 就从左上角开始用 BFS 遍历迷宫图, 逐步计算出它到每个结点的最短路距离 (如图 6-12 (a) 所示), 以及这些最短路径上每个结点的“前一个结点”

(如图 6-12 (b) 所示)。

0	1		11	12
1		9	10	11
2		8		
3		7	8	9
4	5	6		10
5	6	7	8	9

(a) 从左上角出发到各个格子的最短距离



(b) 扩展顺序和父亲指针

图 6-12 用 BFS 求迷宫中最短路

注意,如果把图 6-12 (b) 中的箭头理解成“指向父亲的指针”,那么迷宫中的格子就变成了一棵树——除了起点之外,每个结点恰好有一个父亲。如果看不出来,可以把这棵树画成如图 6-13 所示的样子。这棵树称为最短路树,或者 BFS 树。

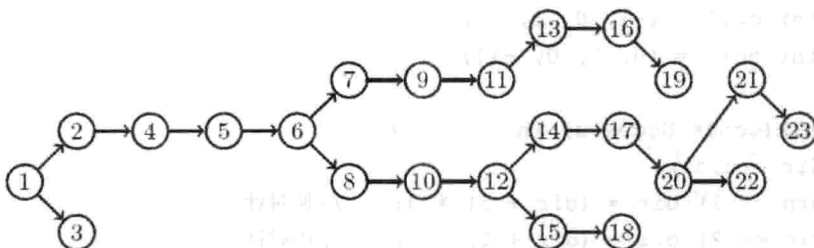


图 6-13 BFS 树的层次画法

例题 6-14 Abbott 的复仇 (Abbott's Revenge, ACM/ICPC World Finals 2000, UVa 816)

有一个最多包含 9×9 个交叉点的迷宫。输入起点、离开起点时的朝向和终点,求一条最短路(多解时任意输出一个即可)。

这个迷宫的特殊之处在于:进入一个交叉点的方向(用 NEWS 这 4 个字母分别表示北东南西,即上右下左)不同,允许出去的方向也不同。例如,1 2 WLF NR ER *表示交叉点(1,2)(上数第 1 行,左数第 2 列)有 3 个路标(字符“*”只是结束标志),如果进入该交叉点时的朝向为 W(即朝左),则可以左转(L)或者直行(F);如果进入时朝向为 N 或者 E 则只能右转(R),如图 6-14 所示。

注意:初始状态是“刚刚离开口”,所以即使出口和入口重合,最短路也不为空。例如,图 6-14 中的一条最短路为 (3,1) (2,1) (1,1) (1,2) (2,2) (2,3) (1,3) (1,2) (1,1) (2,1) (2,2) (1,2) (1,3) (2,3) (3,3)。

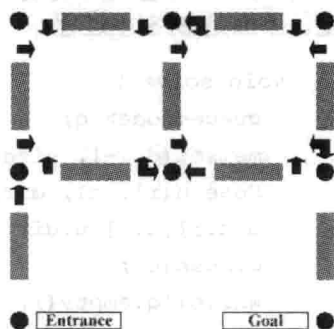


图 6-14 迷宫及走向

【分析】

本题和普通的迷宫在本质上是一样的,但是由于“朝向”也起到了关键作用,所以需要用三元组 (r, c, dir) 表示“位于 (r, c) , 面朝 dir ”这个状态。假设入口位置为 (r_0, c_0) , 朝向为 dir , 则初始状态并不是 (r_0, c_0, dir) , 而是 (r_1, c_1, dir) , 其中, (r_1, c_1) 是 (r_0, c_0) 沿着方向 dir 走一步之后的坐标。此处用 $d[r][c][dir]$ 表示初始状态到 (r, c, dir) 的最短路长度, 并且用 $p[r][c][dir]$ 保存了状态 (r, c, dir) 在 BFS 树中的父结点。

提示 6-22: 很多复杂的迷宫问题都可以转化为最短路问题, 然后用 BFS 求解。在套用 BFS 框架之前, 需要先搞清楚图中的“结点”包含哪些内容。

代码比较长, 下面一点一点地分析。首先是输入过程。将 4 个方向和 3 种“转弯方式”编号为 0~3 和 0~2, 并且提供相应的转换函数:

```
const char* dirs = "NESW";           //顺时针旋转
const char* turns = "FLR";
int dir_id(char c) { return strchr(dirs, c) - dirs; }
int turn_id(char c) { return strchr(turns, c) - turns; }
```

接下来是“行走”函数, 根据当前状态和转弯方式, 计算出后继状态:

```
const int dr[] = {-1, 0, 1, 0};
const int dc[] = {0, 1, 0, -1};

Node walk(const Node& u, int turn) {
    int dir = u.dir;
    if(turn == 1) dir = (dir + 3) % 4;    //逆时针
    if(turn == 2) dir = (dir + 1) % 4;    //顺时针
    return Node(u.r + dr[dir], u.c + dc[dir], dir);
}
```

输入函数比较简单, 作用就是读取 r_0, c_0, dir , 并且计算出 r_1, c_1 , 然后读入 has_edge 数组, 其中 $has_edge[r][c][dir][turn]$ 表示当前状态是 (r, c, dir) , 是否可以沿着转弯方向 $turn$ 行走。下面是 BFS 主过程:

```
void solve() {
    queue<Node> q;
    memset(d, -1, sizeof(d));
    Node u(r1, c1, dir);
    d[u.r][u.c][u.dir] = 0;
    q.push(u);
    while(!q.empty()) {
        Node u = q.front(); q.pop();
        if(u.r == r2 && u.c == c2) { print_ans(u); return; }
        for(int i = 0; i < 3; i++) {
            Node v = walk(u, i);
```

```

    if(has_edge[u.r][u.c][u.dir][i] && inside(v.r, v.c)
    && d[v.r][v.c][v.dir] < 0) {
        d[v.r][v.c][v.dir] = d[u.r][u.c][u.dir] + 1;
        p[v.r][v.c][v.dir] = u;
        q.push(v);
    }
}
printf("No Solution Possible\n");
}

```

最后是解的打印过程。它也可以写成递归函数，不过用 `vector` 保存结点可以避免递归时出现栈溢出，并且更加灵活。

提示 6-23: 使用 BFS 求出图的最短路之后，可以用递归方式打印最短路的具体路径。如果最短路非常长，递归可能会引起栈溢出，此时可以改用循环，用 `vector` 保存路径。

```

void print_ans(Node u) {
    //从目标结点逆序追溯到初始结点
    vector<Node> nodes;
    for(;;) {
        nodes.push_back(u);
        if(d[u.r][u.c][u.dir] == 0) break;
        u = p[u.r][u.c][u.dir];
    }
    nodes.push_back(Node(r0, c0, dir));

    //打印解，每行 10 个
    int cnt = 0;
    for(int i = nodes.size()-1; i >= 0; i--) {
        if(cnt % 10 == 0) printf(" ");
        printf(" (%d,%d)", nodes[i].r, nodes[i].c);
        if(++cnt % 10 == 0) printf("\n");
    }
    if(nodes.size() % 10 != 0) printf("\n");
}

```

本题非常重要，强烈建议读者搞懂所有细节，并能独立编写程序。

6.4.3 拓扑排序

例题 6-15 给任务排序 (Ordering Tasks, UVa 10305)

假设有 n 个变量，还有 m 个二元组 (u, v) ，分别表示变量 u 小于 v 。那么，所有变量从小到大排列起来应该是什么样子的呢？例如，有 4 个变量 a, b, c, d ，若已知 $a < b, c < b, d < c$,

则这4个变量的排序可能是 $a < d < c < b$ 。尽管还有其他可能（如 $d < a < c < b$ ），你只需找出其中一个即可。

【分析】

把每个变量看成一个点，“小于”关系看成有向边，则得到了一个有向图。这样，我们的任务实际上是把一个图的所有结点排序，使得每一条有向边 (u, v) 对应的 u 都排在 v 的前面。在图论中，这个问题称为拓扑排序（topological sort）。

不难发现：如果图中存在有向环，则不存在拓扑排序，反之则存在。不包含有向环的有向图称为有向无环图（Directed Acyclic Graph, DAG）。可以借助 DFS 完成拓扑排序：在访问完一个结点之后把它加到当前拓扑序的首部（想一想，为什么不是尾部）。

```
int c[maxn];
int topo[maxn], t;
bool dfs(int u){
    c[u] = -1; //访问标志
    for(int v = 0; v < n; v++) if(G[u][v]) {
        if(c[v]<0) return false; //存在有向环，失败退出
        else if(!c[v] && !dfs(v)) return false;
    }
    c[u] = 1; topo[--t]=u;
    return true;
}
bool toposort(){
    t = n;
    memset(c, 0, sizeof(c));
    for(int u = 0; u < n; u++) if(!c[u])
        if(!dfs(u)) return false;
    return true;
}
```

这里用到了一个 c 数组， $c[u]=0$ 表示从来没有访问过（从来没有调用过 $\text{dfs}(u)$ ）； $c[u]=1$ 表示已经访问过，并且还递归访问过它的所有子孙（即 $\text{dfs}(u)$ 曾被调用过，并已返回）； $c[u]=-1$ 表示正在访问（即递归调用 $\text{dfs}(u)$ 正在栈帧中，尚未返回）。

提示 6-24：可以用 DFS 求出有向无环图（DAG）的拓扑排序。如果排序失败，说明该有向图存在有向环，不是 DAG。

6.4.4 欧拉回路

有一条名为 Pregel 的河流经过 Königsberg 城。城中有 7 座桥，把河中的两个岛与河岸连接起来。当地居民热衷于一个难题：是否存在一条路线，可以不重复地走遍 7 座桥。这就是著名的七桥问题。它由大数学家欧拉首先提出，并给出了完美的解答，如图 6-15 所示。

欧拉首先把图 6-15 (a) 中的七桥问题用图论的语言改写成图 6-15 (b)，则问题变成

了：能否从无向图中的一个结点出发走出一条道路，每条边恰好经过一次。这样的路线称为欧拉道路（eulerian path），也可以形象地称为“一笔画”。

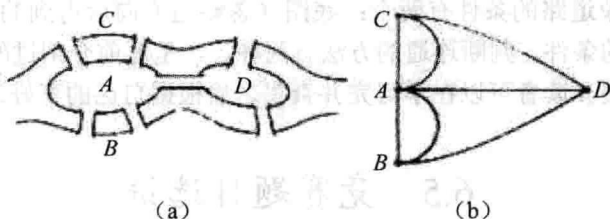


图 6-15 七桥问题

不难发现，在欧拉道路中，“进”和“出”是对应的——除了起点和终点外，其他点的“进出”次数应该相等。换句话说，除了起点和终点外，其他点的度数（degree）应该是偶数。很可惜，在七桥问题中，所有 4 个点的度数均是奇数（这样的点也称奇点），因此不可能存在欧拉道路。上述条件也是充分条件——如果一个无向图是连通的，且最多只有两个奇点，则一定存在欧拉道路。如果有两个奇点，则必须从其中一个奇点出发，另一个奇点终止；如果奇点不存在，则可以从任意点出发，最终一定会回到该点（称为欧拉回路）。

用类似的推理方式可以得到有向图的结论：最多只能有两个点的入度不等于出度，而且必须是其中一个点的出度恰好比入度大 1（把它作为起点），另一个的入度比出度大 1（把它作为终点）。当然，还有一个前提条件：在忽略边的方向后，图必须是连通的。

下面是程序，它同时适用于欧拉道路和回路。但如果需要打印的是欧拉道路，在主程序中调用时，参数必须是道路的起点。另外，打印的顺序是逆序的，因此在真正使用这份代码时，应当把 printf 语句替换成一条 push 语句，把边(u,v)压入一个栈内。

```
void euler(int u){
    for(int v = 0; v < n; v++) if(G[u][v] && !vis[u][v]){
        vis[u][v] = vis[v][u] = 1;
        euler(v);
        printf("%d %d\n", u, v);
    }
}
```

尽管上面的代码只适用于无向图，但不难改成有向图：把 $\text{vis}[u][v] = \text{vis}[v][u] = 1$ 改成 $\text{vis}[u][v]$ 即可。

提示 6-25：根据连通性和度数可以判断出无向图和有向图是否存在欧拉道路和欧拉回路。可以用 DFS 构造欧拉回路和欧拉道路。

例题 6-16 单词（Play On Words, UVa 10129）

输入 n ($n \leq 100000$) 个单词，是否可以把所有这些单词排成一个序列，使得每个单词的第一个字母和上一个单词的最后一个字母相同（例如 acm、malform、mouse）。每个单词最多包含 1000 个小写字母。输入中可以有重复单词。